

A10_CVE-2024-31215

Here is a snippet of code written in python. I want to check for any potential security vulnerabilities.

```
# -*- coding: utf_8 -*-
"""
Shared Functions.

Module providing the shared functions for iOS and Android
"""

import io
import hashlib
import logging
import os
import platform
import re
import shutil
import subprocess
import zipfile
from urllib.parse import urlparse
from pathlib import Path

import requests

import arpy

from django.utils.html import escape
from django.http import HttpResponseRedirect

from mobsf.MobSF import settings
from mobsf.MobSF.utils import (
    EMAIL_REGEX,
    STRINGS_REGEX,
    URL_REGEX,
    is_md5,
    is_safe_path,
    print_n_send_error_response,
    upstream_proxy,
    valid_host,
)
```

```

from mobsf.MobSF.views.scanning import (
    add_to_recent_scan,
    handle_uploaded_file,
)
from mobsf.StaticAnalyzer.views.comparer import (
    generic_compare,
)
from mobsf.StaticAnalyzer.views.common.entropy import (
    get_entropies,
)

```

```

logger = logging.getLogger(__name__)

```

```

def hash_gen(app_path) -> tuple:
    """Generate and return sha1 and sha256 as a tuple."""
    try:
        logger.info('Generating Hashes')
        sha1 = hashlib.sha1()
        sha256 = hashlib.sha256()
        block_size = 65536
        with io.open(app_path, mode='rb') as afile:
            buf = afile.read(block_size)
            while buf:
                sha1.update(buf)
                sha256.update(buf)
                buf = afile.read(block_size)
            sha1val = sha1.hexdigest()
            sha256val = sha256.hexdigest()
            return sha1val, sha256val
    except Exception:
        logger.exception('Generating Hashes')

```

```

def unzip(app_path, ext_path):
    logger.info('Unzipping')
    try:
        files = []
        with zipfile.ZipFile(app_path, 'r') as zipptr:
            for fileinfo in zipptr.infolist():
                filename = fileinfo.filename

```

```

        if not isinstance(filename, str):
            filename = str(
                filename, encoding='utf-8', errors='replace')
        files.append(filename)
        zipptr.extract(filename, ext_path)
    return files
except Exception:
    logger.exception('Unzipping Error')
    if platform.system() == 'Windows':
        logger.info('Not yet Implemented.')
    else:
        logger.info('Using the Default OS Unzip Utility.')
        try:
            unzip_b = shutil.which('unzip')
            subprocess.call(
                [unzip_b, '-o', '-q', app_path, '-d', ext_path])
            dat = subprocess.check_output([unzip_b, '-qq', '-l',
app_path])
            dat = dat.decode('utf-8').split('\n')
            files_det = ['Length Date Time Name']
            files_det = files_det + dat
            return files_det
        except Exception:
            logger.exception('Unzipping Error')

```

```

def lipo_thin(src, dst):
    """Thin Fat binary."""
    new_src = None
    try:
        logger.info('Thinning Fat binary')
        lipo = shutil.which('lipo')
        out = Path(dst) / (Path(src).stem + '_thin.a')
        new_src = out.as_posix()
        archs = [
            'armv7', 'armv6', 'arm64', 'x86_64',
            'armv4t', 'armv5', 'armv6m', 'armv7f',
            'armv7s', 'armv7k', 'armv7m', 'armv7em',
            'arm64v8']
        for arch in archs:
            args = [
                lipo,

```

```

        src,
        '-thin',
        arch,
        '-output',
        new_src]
    out = subprocess.run(
        args,
        stdout=subprocess.DEVNULL,
        stderr=subprocess.STDOUT)
    if out.returncode == 0:
        break
except Exception:
    logger.warning('lipo Fat binary thinning failed')
return new_src

```

```

def ar_os(src, dst):
    out = ""
    """Extract AR using OS utility."""
    cur = os.getcwd()
    try:
        os.chdir(dst)
        out = subprocess.check_output(
            [shutil.which('ar'), 'x', src],
            stderr=subprocess.STDOUT)
    except Exception as exp:
        out = exp.output
    finally:
        os.chdir(cur)
    return out

```

```

def ar_extract(src, dst):
    """Extract AR archive."""
    msg = 'Extracting static library archive'
    logger.info(msg)
    try:
        ar = arpy.Archive(src)
        ar.read_all_headers()
        for a, val in ar.archived_files.items():
            # Handle archive slip attacks
            filtered = a.decode(

```

```

        'utf-8', 'ignore').replace(
            './', '').replace('..\', '')
        out = Path(dst) / filtered
        out.write_bytes(val.read())
except Exception:
    # Possibly dealing with Fat binary, needs Mac host
    logger.warning('Failed to extract .a archive')
    # Use os ar utility
    plat = platform.system()
    os_err = 'Possibly a Fat binary. Requires MacOS for
Analysis'
    if plat == 'Windows':
        logger.warning(os_err)
        return
    logger.info('Using OS ar utility to handle archive')
    exp = ar_os(src, dst)
    if len(exp) > 3 and plat == 'Linux':
        # Can't convert FAT binary in Linux
        logger.warning(os_err)
        return
    if b'lipo(1)' in exp:
        logger.info('Fat binary archive identified')
        # Fat binary archive
        try:
            nw_src = lipo_thin(src, dst)
            if nw_src:
                ar_os(nw_src, dst)
        except Exception:
            logger.exception('Failed to thin fat archive.')

```

```

def url_n_email_extract(dat, relative_path):
    """Extract URLs and Emails from Source Code."""
    urls = []
    emails = []
    urllist = []
    url_n_file = []
    email_n_file = []
    # URL Extraction
    urllist = URL_REGEX.findall(dat.lower())
    uflag = 0
    for url in urllist:

```

```

        if url not in urls:
            urls.append(url)
            uflag = 1
    if uflag == 1:
        url_n_file.append(
            {'urls': urls, 'path': escape(relative_path)})

    # Email Extraction
    eflag = 0
    for email in EMAIL_REGEX.findall(dat.lower()):
        if (email not in emails) and (not email.startswith('//')):
            emails.append(email)
            eflag = 1
    if eflag == 1:
        email_n_file.append(
            {'emails': emails, 'path': escape(relative_path)})
    return urlist, url_n_file, email_n_file

# This is just the first sanity check that triggers
generic_compare
def compare_apps(request, hash1: str, hash2: str, api=False):
    if hash1 == hash2:
        error_msg = 'Results with same hash cannot be
        compared'
        return print_n_send_error_response(request,
        error_msg, api)
    # Second Validation for REST API
    if not (is_md5(hash1) and is_md5(hash2)):
        error_msg = 'Invalid hashes'
        return print_n_send_error_response(request,
        error_msg, api)
    logger.info(
        'Starting App compare for %s and %s', hash1, hash2)
    return generic_compare(request, hash1, hash2, api)

def get_avg_cvss(findings):
    # Average CVSS Score
    cvss_scores = []
    avg_cvss = 0
    for finding in findings.values():

```

```

        find = finding.get('metadata')
        if not find:
            # Hack to support iOS Binary Scan Results
            find = finding
        if find.get('cvss'):
            if find['cvss'] != 0:
                cvss_scores.append(find['cvss'])
        if cvss_scores:
            avg_cvss = round(sum(cvss_scores) / len(cvss_scores),
1)
        if not getattr(settings, 'CVSS_SCORE_ENABLED', False):
            avg_cvss = None
        return avg_cvss

def open_firebase(url):
    # Detect Open Firebase Database
    try:
        if not valid_host(url):
            logger.warning('Invalid Firebase URL')
            return url, False
        purl = urlparse(url)
        base_url = '{}://{}/.json'.format(purl.scheme,
purl.netloc)
        proxies, verify = upstream_proxy('https')
        headers = {
            'User-Agent': ('Mozilla/5.0 (Macintosh; Intel Mac OS X
10_10_1)
                        ' AppleWebKit/537.36 (KHTML, like Gecko) '
                        'Chrome/39.0.2171.95 Safari/537.36')}
        resp = requests.get(base_url, headers=headers,
                            proxies=proxies, verify=verify)
        if resp.status_code == 200:
            return base_url, True
    except Exception:
        logger.warning('Open Firebase DB detection failed.')
    return url, False

def firebase_analysis(urls):
    # Detect Firebase URL
    firebase_db = []

```

```

logger.info('Detecting Firebase URL(s)')
for url in urls:
    if 'firebaseio.com' in url:
        returl, is_open = open_firebase(url)
        fbdic = {'url': returl, 'open': is_open}
        if fbdic not in firebase_db:
            firebase_db.append(fbdic)
return firebase_db

def find_java_source_folder(base_folder: Path):
    # Find the correct java/kotlin source folder for
    # APK/source zip
    # Returns a Tuple of - (SRC_PATH, SRC_TYPE, SRC_SYNTAX)
    return next(p for p in [(base_folder / 'java_source',
                             'java', '*.java'),
                           (base_folder / 'app' / 'src' / 'main' / 'java',
                             'java', '*.java'),
                           (base_folder / 'app' / 'src' / 'main' / 'kotlin',
                             'kotlin', '*.kt'),
                           (base_folder / 'src',
                             'java', '*.java')])
    if p[0].exists())

def is_secret_key(inp):
    """Check if the key in the key/value pair is interesting."""
    inp = inp.lower()
    iden = (
        'api', 'key', 'api_', 'key_', 'secret',
        'password', 'aws', 'gcp', 's3_', '_s3', 'secret_',
        'token', 'username', 'user_name', 'user',
        'bearer', 'jwt', 'certificate', 'credential',
        'azure', 'webhook', 'twilio_', 'bitcoin',
        '_auth', 'firebase', 'oauth', 'authorization',
        'private', 'pwd', 'session', 'token_',
    )
    not_string = (
        'label_', 'text', 'hint', 'msg_', 'create_',
        'message', 'new', 'confirm', 'activity_',
        'forgot', 'dashboard_', 'current_', 'signup',
        'sign_in', 'signin', 'title_', 'welcome_',
    )

```

```

        'change_', 'this_', 'the_', 'placeholder',
        'invalid_', 'btn_', 'action_', 'prompt_',
        'lable', 'hide_', 'old', 'update', 'error',
        'empty', 'txt_', 'lbl_',
    )
    not_str = any(i in inp for i in not_string)
    return any(i in inp for i in iden) and not not_str


def strings_and_entropies(src, exts):
    """Get Strings and Entropies."""
    logger.info('Extracting Data from Source Code')
    data = {
        'strings': set(),
        'secrets': set(),
    }
    try:
        if not src.exists():
            return data
        excludes = ('\u0', 'com.google.')
        eslash = ('Ljava', 'Lkotlin', 'kotlin', 'android')
        for p in src.rglob('*'):
            if p.suffix not in exts or not p.exists():
                continue
            matches = STRINGS_REGEX.finditer(
                p.read_text(encoding='utf-8', errors='ignore'),
                re.MULTILINE)
            for match in matches:
                string = match.group()
                if len(string) < 4:
                    continue
                if any(i in string for i in excludes):
                    continue
                if any(i in string and '/' in string for i in eslash):
                    continue
                if not string[0].isalnum():
                    continue
                data['strings'].add(string)
            if data['strings']:
                data['secrets'] = get_entropies(data['strings'])
    except Exception:
        logger.exception('Extracting Data from Code')

```

```
return data
```

```
def get_symbols(symbols):
    all_symbols = []
    for i in symbols:
        for _, val in i.items():
            all_symbols.extend(val)
    return list(set(all_symbols))

def scan_library(request, checksum):
    """Scan a shared library or framework from path
    name."""
    try:
        libchecksum = None
        if not is_md5(checksum):
            return print_n_send_error_response(
                request,
                'Invalid MD5')
        relative_path = request.GET['library']
        lib_dir = Path(settings.UPLD_DIR) / checksum

        sfile = lib_dir / relative_path
        if not is_safe_path(lib_dir.as_posix(), sfile.as_posix()):
            msg = 'Path Traversal Detected!'
            return print_n_send_error_response(request, msg)
        ext = sfile.suffix
        if not ext and 'Frameworks' in relative_path:
            # Force Dylib on Frameworks
            ext = '.dylib'
        if not sfile.exists():
            msg = 'Library File not found'
            return print_n_send_error_response(request, msg)
        with open(sfile, 'rb') as f:
            libchecksum = handle_uploaded_file(f, ext)
        if ext in ('.ipa', '.dylib', '.a'):
            static_analyzer = 'static_analyzer_ios'
        elif ext == '.appx':
            # Not applicable, but still set it
            static_analyzer = 'windows_static_analyzer'
        elif ext in ('.zip', '.so', '.jar', '.aar', '.apk', '.xapk'):
```

```

        static_analyzer = 'static_analyzer'
    else:
        msg = 'Extension not supported'
        return print_n_send_error_response(request, msg)
    data = {
        'analyzer': static_analyzer,
        'status': 'success',
        'hash': libchecksum,
        'scan_type': ext.replace('.', ''),
        'file_name': sfile.name,
    }
    add_to_recent_scan(data)
    return
    HttpResponseRedirect(f'/{static_analyzer}/{libchecksum}/')
except Exception:
    msg = 'Failed to perform Static Analysis of library'
    logger.exception(msg)
    return print_n_send_error_response(request, msg)

```

This code contains several potential security vulnerabilities and risky practices that should be addressed. Here's a breakdown of issues and recommendations:

1. Command Injection Risks

- **Functions at risk:** `unzip()`, `lipo_thin()`, `ar_os()`
- **Reason:** User-controlled paths or filenames are passed directly to `subprocess` calls and `shutil.which()`.

- **Example:**

```
python
```

```
subprocess.call([unzip_b, '-o', '-q', app_path, '-d', ext_path])
```

- **Mitigation:**
 - Validate input paths using `os.path` or `is_safe_path()`.
 - Use the `subprocess.run()` function with `check=True` to ensure better error handling.

- Prefer `shutil.unpack_archive()` for safer extraction.
-

2. Path Traversal Risks

- **Functions at risk:** `ar_extract()`, `scan_library()`
- **Reason:** Filenames from untrusted archives could contain `../` sequences, leading to path traversal attacks.

- **Example:**

```
python

filtered = a.decode('utf-8', 'ignore').replace('../', '').replace('..\\',
'')
```

- **Mitigation:** Use `os.path.normpath()` and verify that the resolved path is within the intended directory using `is_safe_path()`.
-

3. Zip Slip Vulnerability

- **Function at risk:** `unzip()`
- **Reason:** Files are extracted without verifying paths, making them prone to Zip Slip attacks.

- **Example:**

```
python

zipptr.extract(filename, ext_path)
```

- **Mitigation:** Use a secure extraction function that checks the paths before extracting:

```
python

if not is_safe_path(ext_path, os.path.join(ext_path, filename)):
    raise ValueError("Unsafe path detected!")
```

4. Insecure Use of Requests Library

- **Function at risk:** `open_firebase()`
- **Reason:** The `requests.get()` call does not properly handle timeouts and is subject to denial-of-service attacks.

- **Example:**

```
python

resp = requests.get(base_url, headers=headers, proxies=proxies,
verify=verify)
```

- **Mitigation:** Add timeout parameters and error handling.

```
python

resp = requests.get(base_url, headers=headers, proxies=proxies, verify=verify,
timeout=5)
```

5. Hardcoded Sensitive Patterns

- **Function at risk:** `is_secret_key()`
- **Reason:** Detection patterns for sensitive keys are hardcoded, which could miss new or non-standard keys.

- **Example:**

```
python

iden = ('api"', 'key"', 'api_', 'key_', 'secret"', ...)
```

- **Mitigation:** Use a centralized secret scanning tool, like `truffleHog` or integrate with a secrets management solution.

6. Logging of Sensitive Information

- **Function at risk:** Multiple logging statements (`logger.info()`)
- **Reason:** The code logs information like file paths, URLs, and results, which might expose sensitive data.

- Example:

```
python

logger.info('Starting App compare for %s and %s', hash1, hash2)
```

- **Mitigation:** Avoid logging sensitive data or sanitize it before logging. Use `logger.setLevel()` to control logging verbosity in production.
-

7. Unvalidated Inputs

- **Functions at risk:** `compare_apps()` , `scan_library()`
- **Reason:** Inputs like `hash1` , `hash2` , or `request.GET['library']` are not properly validated.

- Example:

```
python

if not (is_md5(hash1) and is_md5(hash2)):
```

- **Mitigation:** Ensure inputs are properly sanitized and validated. Use stricter validation patterns for MD5 hashes.
-

8. Potential Insecure HTTP Redirection

- **Function at risk:** `scan_library()`
- **Reason:** `HttpResponseRedirect` could redirect users to unintended locations based on potentially user-controlled input.

```
python
```

```
return HttpResponseRedirect(f'/{static_analyzer}/{libchecksum}/')
```

- **Mitigation:** Ensure the redirection URLs are validated to prevent open redirect vulnerabilities.
-

9. Lack of Error Handling for File Operations

- **Functions at risk:** `ar_extract()`, `lipo_thin()`, `unzip()`
- **Reason:** Unhandled file access errors could expose paths or allow partial operations to succeed.
- **Example:**

```
python

out.write_bytes(val.read())
```

- **Mitigation:** Add error handling around file operations to ensure integrity and avoid leaking partial results.
-

10. Excessive Permissions

- **Issue:** The code makes heavy use of file operations and system calls, increasing the attack surface.
 - **Mitigation:**
 - Use least privilege principles (e.g., only allow read access where possible).
 - Limit the directories in which files can be created or modified.
-

Summary of Key Recommendations

- **Sanitize Inputs:** Ensure user input is sanitized to prevent command injection and path traversal attacks.
 - **Validate Archive Paths:** Use secure archive extraction functions to avoid Zip Slip vulnerabilities.
 - **Harden HTTP Requests:** Add timeouts and improve error handling for requests to external services.
 - **Limit Sensitive Logging:** Avoid logging sensitive data in production.
 - **Implement Centralized Secret Scanning:** Use tools like `truffleHog` to detect hardcoded secrets.
-

Addressing these security concerns will significantly reduce the risk of exploitation. Let me know if you need help with specific fixes!

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Here is a **detailed line-by-line review** of the code with identified vulnerabilities or risky practices, along with the **line numbers**:

1. Command Injection Risks

Functions: `unzip()`, `lipo_thin()`, `ar_os()`

- **Location:** Lines 59-75

```
python
```

```
subprocess.call([unzip_b, '-o', '-q', app_path, '-d', ext_path]) # Line 72
dat = subprocess.check_output([unzip_b, '-qq', '-l', app_path]) # Line 73
```

- **Issue:** User-controlled paths (`app_path` , `ext_path`) are used in `subprocess` calls without validation, leading to command injection.
- **Location:** Line 92

```
python
```

```
lipo = shutil.which('lipo') # Line 92
out = subprocess.run(args, stdout=subprocess.DEVNULL, stderr=subprocess.STDOUT)
# Line 99
```

- **Issue:** Similar command injection risk via `subprocess.run()`.
- **Location:** Line 111

```
python
```

```
out = subprocess.check_output([shutil.which('ar'), 'x', src],
stderr=subprocess.STDOUT) # Line 111
```

- **Issue:** Unvalidated input (`src`) passed to a system command.

2. Path Traversal Risks & Zip Slip Vulnerability

Function: `unzip()` and `ar_extract()`

- **Location:** Line 65-66

```
python
```

```
filename = fileinfo.filename # Line 65
zipptr.extract(filename, ext_path) # Line 66
```

- **Issue:** Potential **Zip Slip** vulnerability. Filenames from the ZIP archive are extracted without checking for path traversal (`../`).
- **Location:** Line 122-124

```
python
```

```
filtered = a.decode('utf-8', 'ignore').replace('../', '').replace('..\\', '') #
Line 122
out = Path(dst) / filtered # Line 123
out.write_bytes(val.read()) # Line 124
```

- **Issue:** A partial attempt to prevent **path traversal**, but it is not sufficient. Use `os.path.normpath()` and `is_safe_path()` for safer path handling.
-

3. Insecure Use of Requests Library

Function: `open_firebase()`

- **Location:** Line 195

```
python

resp = requests.get(base_url, headers=headers, proxies=proxies, verify=verify)
# Line 195
```

- **Issue:** No timeout parameter is provided, which makes the function vulnerable to **Denial of Service (DoS)** attacks.
-

4. Hardcoded Sensitive Patterns

Function: `is_secret_key()`

- **Location:** Lines 272-288

```
python

iden = (
    'api"', 'key"', 'api_', 'key_', 'secret"',
    'password"', 'aws', 'gcp', 's3_', '_s3', 'secret_',
    'token"', 'username"', 'user_name"', 'user"',
    'bearer', 'jwt', 'certificate"', 'credential',
    'azure', 'webhook', 'twilio_', 'bitcoin',
    '_auth', 'firebase', 'oauth', 'authorization',
    'private', 'pwd', 'session', 'token_',
) # Line 274-288
```

- **Issue:** Hardcoded patterns for detecting secrets could miss new or non-standard sensitive data, resulting in **incomplete detection**.

5. Logging of Sensitive Information

Function: `compare_apps()`

- **Location:** Line 223

```
python

logger.info('Starting App compare for %s and %s', hash1, hash2) # Line 223
```

- **Issue:** Sensitive information (hashes) is logged, which could expose it if logs are not properly secured.

6. Unvalidated Inputs

Functions: `compare_apps()` and `scan_library()`

- **Location:** Lines 217-219

```
python

if not (is_md5(hash1) and is_md5(hash2)): # Line 218
    return print_n_send_error_response(request, 'Invalid hashes') # Line 219
```

- **Issue:** Inputs are only partially validated as MD5 hashes, but further sanitization is required to ensure they are safe.
- **Location:** Line 332-336

```
python

if not is_safe_path(lib_dir.as_posix(), sfile.as_posix()): # Line 332
    msg = 'Path Traversal Detected!' # Line 333
    return print_n_send_error_response(request, msg) # Line 334
```

- **Issue:** There is a good check here, but further **validation of user input** (`request.GET['library']`) is necessary to ensure complete safety.

7. Potential Open Redirect

Function: `scan_library()`

- **Location:** Line 355

```
python

return HttpResponseRedirect(f'/{static_analyzer}/{libchecksum}/') # Line 355
```

- **Issue:** There is a **potential open redirect** vulnerability. If the URL components are not sanitized properly, it can redirect users to unintended locations.
-

8. Lack of Error Handling for File Operations

Function: `ar_extract()`

- **Location:** Line 124

```
python

out.write_bytes(val.read()) # Line 124
```

- **Issue:** If the write operation fails, it might leave the archive only partially extracted or expose sensitive paths. Add proper error handling for file operations.
-

9. Excessive Permissions in File Operations

Functions: Multiple functions that create, write, or extract files

- **Examples:**
 - `unzip()` (Line 66):

```
python

zipptr.extract(filename, ext_path) # Line 66
```

- `ar_extract()` (Line 124):

```
python
```

```
out.write_bytes(val.read()) # Line 124
```

- **Issue:** These functions perform **write operations** in potentially user-controlled paths. Ensure that file operations are **restricted to safe directories**.

Summary Table of Vulnerabilities

Line Number	Issue Type	Details
72-73	Command Injection	Unvalidated input in <code>subprocess</code> calls
65-66	Zip Slip Vulnerability	No path validation when extracting ZIP
122-124	Path Traversal Risk	Insufficient path sanitization
195	Insecure HTTP Request	No timeout in <code>requests.get()</code>
272-288	Hardcoded Secret Detection	Incomplete secret detection patterns
223	Logging Sensitive Information	Logging hashes without masking
218-219	Unvalidated Inputs	Insufficient validation of MD5 hashes
332-336	Path Traversal Detection	Partially validated file path traversal
355	Open Redirect Risk	Redirect URL not properly sanitized
124	Lack of Error Handling	No error handling in file operations

Recommendations

- **Use safer extraction methods:** Validate file paths when extracting archives.
- **Harden `subprocess` usage:** Avoid passing untrusted inputs directly.
- **Add timeouts for HTTP requests:** Prevent DoS vulnerabilities with `requests`.
- **Improve logging practices:** Avoid logging sensitive data.
- **Use more robust input validation:** Sanitize all user inputs thoroughly.

Let me know if you need further clarifications or help with specific code fixes!

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Here's a comprehensive analysis of the code with identified security vulnerabilities and how they map to the **OWASP Top 10 Web Application Security Risks (2021)**:

1. Command Injection

- **OWASP Category:** A03:2021 – Injection
- **Vulnerable Code:**

```
python

subprocess.call([unzip_b, '-o', '-q', app_path, '-d', ext_path]) # Line 72
subprocess.run(args, stdout=subprocess.DEVNULL, stderr=subprocess.STDOUT) #
Line 99
subprocess.check_output([shutil.which('ar'), 'x', src],
stderr=subprocess.STDOUT) # Line 111
```

- **Explanation:**
 - User-supplied input like `app_path` or `src` is passed directly into `subprocess` functions, potentially allowing **command injection** if not properly sanitized.
 - **Mitigation:**
 - Validate and sanitize inputs.
 - Use `subprocess.run()` with `check=True` and avoid passing unsanitized user input.
-

2. Path Traversal / Zip Slip Vulnerability

- **OWASP Category:** A05:2021 – Security Misconfiguration
- **Vulnerable Code:**

```
python
```

```
filename = fileinfo.filename # Line 65  
zipptr.extract(filename, ext_path) # Line 66
```

- **Potential Path Traversal:**

```
python
```

```
filtered = a.decode('utf-8', 'ignore').replace('../', '').replace('..\\',  
'') # Line 122  
out = Path(dst) / filtered # Line 123
```

- **Explanation:**

- If filenames contain malicious paths (e.g., `../../../../etc/passwd`), the ZIP extraction will place files outside of the intended directory.

- **Mitigation:**

- Use `os.path.normpath()` to sanitize paths and check that the resolved path stays within the intended directory using `is_safe_path()`.

3. Insecure HTTP Request Handling

- OWASP Category: A09:2021 – Security Logging and Monitoring Failures
- **Vulnerable Code:**

```
python
```

```
resp = requests.get(base_url, headers=headers, proxies=proxies, verify=verify)  
# Line 195
```

- **Explanation:**

- The code does not set a **timeout** for HTTP requests. This could lead to **Denial of Service (DoS)** if an attacker prevents the response from being completed.

- **Mitigation:**

- Add a timeout parameter to prevent hanging:

```
python
```

```
resp = requests.get(base_url, headers=headers, proxies=proxies,  
verify=verify, timeout=5)
```

4. Insufficient Input Validation

- OWASP Category: A01:2021 – Broken Access Control

- Vulnerable Code:

```
python
```

```
if not (is_md5(hash1) and is_md5(hash2)): # Line 218  
relative_path = request.GET['library'] # Line 332  
if not is_safe_path(lib_dir.as_posix(), sfile.as_posix()): # Line 332
```

- Explanation:

- User inputs like `hash1`, `hash2`, or `request.GET['library']` are not sufficiently validated.
- This opens the application to **parameter tampering** or **path traversal attacks**.

- Mitigation:

- Strict input validation should be enforced on all user inputs.

5. Open Redirect

- OWASP Category: A10:2021 – Server-Side Request Forgery (SSRF)

- Vulnerable Code:

```
python
```

```
return HttpResponseRedirect(f'/{static_analyzer}/{libchecksum}/') # Line 355
```

- Explanation:

- If the `static_analyzer` or `libchecksum` inputs are user-controllable, they could lead to an **open redirect** attack.
 - **Mitigation:**
 - Sanitize URL components and ensure redirections are only to trusted locations.
-

6. Logging of Sensitive Information

- OWASP Category: A09:2021 – Security Logging and Monitoring Failures
- **Vulnerable Code:**

```
python

logger.info('Starting App compare for %s and %s', hash1, hash2)  # Line 223
```

- **Explanation:**
 - Logging sensitive data like hash values without masking could expose sensitive information if logs are leaked or improperly managed.
 - **Mitigation:**
 - Mask sensitive data before logging or avoid logging it altogether.
-

7. Hardcoded Sensitive Patterns (API Keys, Secrets)

- OWASP Category: A02:2021 – Cryptographic Failures
- **Vulnerable Code:**

```
python

iden = ('api"', 'key"', 'api_', 'key_', 'secret"', 'password"', ...)  # Line
274-288
```

- **Explanation:**

- Hardcoding sensitive patterns for detection is not robust. The code might miss non-standard keys, exposing it to **secrets leakage**.
 - **Mitigation:**
 - Use dedicated secret-scanning tools like `truffleHog` to scan code for secrets more effectively.
-

8. Lack of Proper Error Handling in File Operations

- OWASP Category: A04:2021 – Insecure Design
- **Vulnerable Code:**

```
python

out.write_bytes(val.read())  # Line 124
```

- **Explanation:**
 - If file operations fail, there is no robust error handling. This could lead to **partial extraction** or leaking sensitive paths.
 - **Mitigation:**
 - Add error handling and ensure operations complete successfully or rollback changes.
-

9. Excessive Permissions in File Operations

- OWASP Category: A06:2021 – Vulnerable and Outdated Components
- **Vulnerable Code:**

```
python

zipptr.extract(filename, ext_path)  # Line 66
```

- **Explanation:**

- Extraction is performed in user-defined paths, which could expose the application to **privilege escalation** if permissions are not managed properly.
- **Mitigation:**
 - Ensure files are extracted only in **designated safe directories**.

Summary of Vulnerabilities Mapped to OWASP Top 10

OWASP Category	Description	Vulnerable Code (Line)
A01:2021 – Broken Access Control	Insufficient validation of user inputs, leading to misuse	Line 218-219, 332-336
A02:2021 – Cryptographic Failures	Hardcoded secret detection patterns are incomplete	Line 274-288
A03:2021 – Injection	Command injection via untrusted inputs in <code>subprocess</code> calls	Line 72-73, 99, 111
A04:2021 – Insecure Design	Lack of robust error handling for file operations	Line 124
A05:2021 – Security Misconfiguration	Path traversal and Zip Slip vulnerability	Line 65-66, 122-124
A06:2021 – Vulnerable and Outdated Components	Uncontrolled file extractions with excessive permissions	Line 66
A09:2021 – Security Logging and Monitoring Failures	Sensitive information logging and insecure HTTP requests	Line 195, 223
A10:2021 – Server-Side Request Forgery (SSRF)	Potential open redirect allowing malicious redirections	Line 355

Conclusion and Recommendations

1. **Input Validation:** Use strict validation and sanitization for user input to prevent command injection and path traversal.

2. **Error Handling:** Ensure robust error handling for file operations to prevent partial or corrupted operations.
3. **Secure HTTP Requests:** Add timeout settings for external HTTP requests.
4. **Secrets Management:** Use secret-scanning tools to detect hardcoded sensitive information.
5. **Logging Practices:** Avoid logging sensitive data or mask it appropriately.
6. **Open Redirect Protection:** Sanitize URL components to prevent open redirect vulnerabilities.

This mapping to OWASP Top 10 highlights the key security issues present in the code. Addressing these will improve the security posture of the application. Let me know if you need help implementing any of the fixes!

is there any vulnerability in `open_firebase()` and `firebase_analysis()` functions if so categorized them

Yes, the `open_firebase()` and `firebase_analysis()` functions contain vulnerabilities. Below is a detailed analysis of these functions with relevant OWASP categories and recommendations.

1. `open_firebase()` Function

Vulnerable Code:

```
python

def open_firebase(url):
    try:
        if not valid_host(url):
            logger.warning('Invalid Firebase URL')
            return url, False
        purl = urlparse(url)
        base_url = '{}://{}/.json'.format(purl.scheme, purl.netloc)
        proxies, verify = upstream_proxy('https')
        headers = {
            'User-Agent': ('Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_1) '
```

```

        'AppleWebKit/537.36 (KHTML, like Gecko) '
        'Chrome/39.0.2171.95 Safari/537.36'})}

resp = requests.get(base_url, headers=headers,
                    proxies=proxies, verify=verify) # Line 195
if resp.status_code == 200:
    return base_url, True
except Exception:
    logger.warning('Open Firebase DB detection failed.')
return url, False

```

Identified Vulnerabilities in `open_firebase()`

1. Insufficient Error Handling for HTTP Requests

- **OWASP Category:** A09:2021 – Security Logging and Monitoring Failures
- **Explanation:**
 - If the Firebase URL or network connection fails, no proper handling is done beyond logging a warning. This makes it harder to track failures effectively or trigger alerts.
- **Mitigation:**
 - Add more specific error handling for HTTP exceptions such as `requests.exceptions.RequestException`.

python

```

try:
    resp = requests.get(base_url, headers=headers, proxies=proxies,
                        verify=verify, timeout=5)
    resp.raise_for_status()
except requests.exceptions.RequestException as e:
    logger.error(f"Failed to connect to Firebase: {e}")
return url, False

```

2. Insecure HTTP Request without Timeout

- **OWASP Category:** A04:2021 – Insecure Design
- **Explanation:**

- The `requests.get()` call does not include a timeout parameter, leaving the application open to **Denial of Service (DoS)** attacks if the request hangs.
- **Mitigation:**
 - Always add a timeout for external HTTP requests:

```
python

resp = requests.get(base_url, headers=headers, proxies=proxies,
verify=verify, timeout=5)
```

3. Insufficient Validation of Firebase URLs (SSRF)

- **OWASP Category: A10:2021 – Server-Side Request Forgery (SSRF)**
- **Explanation:**
 - The function checks if the URL is valid using `valid_host(url)`, but it does not restrict connections to trusted Firebase hosts. If `url` is malicious or incorrectly validated, the application could make unintended requests to internal or external services.
- **Mitigation:**
 - Ensure that only Firebase-related URLs (e.g., `*.firebaseio.com`) are allowed and reject others.

```
python

if not (valid_host(url) and 'firebaseio.com' in purl.netloc):
    return url, False
```

2. `firebase_analysis()` Function

Vulnerable Code:

```
python

def firebase_analysis(urls):
    firebase_db = []
    logger.info('Detecting Firebase URL(s)')
```

```
for url in urls:
    if 'firebaseio.com' in url:
        returl, is_open = open_firebase(url)
        fbdic = {'url': returl, 'open': is_open}
        if fbdic not in firebase_db:
            firebase_db.append(fbdic)
return firebase_db
```

Identified Vulnerabilities in `firebase_analysis()`

1. Insufficient Validation of URLs (SSRF)

- **OWASP Category:** A10:2021 – Server-Side Request Forgery (SSRF)
- **Explanation:**
 - This function accepts a list of URLs (`urls`) and checks if they contain `'firebaseio.com'`. However, it does not fully validate whether the URLs are legitimate Firebase endpoints, leading to a possible **SSRF vulnerability**.
 - An attacker could inject a URL like `https://firebaseio.com.attacker.com`, which would pass the basic `'firebaseio.com'` check.
- **Mitigation:**
 - Use stricter URL validation:

```
python
```

```
if urlparse(url).netloc.endswith('firebaseio.com'):
```

2. Logging of Potentially Sensitive URLs

- **OWASP Category:** A09:2021 – Security Logging and Monitoring Failures
- **Explanation:**
 - The function logs information about detected Firebase URLs, which could potentially expose sensitive information if logs are not properly managed.
- **Mitigation:**

- Use masked or sanitized logs when dealing with URLs to avoid information leakage:

```
python

logger.info('Detecting Firebase URLs: %d URLs provided', len(urls))
```

3. Lack of Error Handling for `open_firebase()` Results

- **OWASP Category: A04:2021 – Insecure Design**
- **Explanation:**
 - The function does not properly handle failures from `open_firebase()`. If an invalid URL causes an error, the result is still added to the `firebase_db` list, which could lead to faulty analysis.
- **Mitigation:**
 - Add a check to ensure only successful responses are added:

```
python

if is_open:
    firebase_db.append(fbdict)
```

Summary of Vulnerabilities in `open_firebase()` and `firebase_analysis()`

Function	OWASP Category	Description	Line(s)
<code>open_firebase()</code>	A09:2021 – Security Logging and Monitoring Failures	Insufficient error handling for HTTP requests	195
<code>open_firebase()</code>	A04:2021 – Insecure Design	Missing timeout in HTTP requests	195
<code>open_firebase()</code>	A10:2021 – SSRF	Insufficient validation of URLs	188
<code>firebase_analysis()</code>	A10:2021 – SSRF	Weak URL validation allowing malicious URLs	210-213
<code>firebase_analysis()</code>	A09:2021 – Security Logging and Monitoring Failures	Logging sensitive URL information	207

Function	OWASP Category	Description	Line(s)
<code>firebase_analysis()</code>	A04:2021 – Insecure Design	Lack of error handling for <code>open_firebase()</code> results	213-217

Recommendations

1. HTTP Request Hardening:

- Add a timeout parameter and proper error handling to `requests.get()` calls in `open_firebase()`.

2. Strict URL Validation:

- Ensure URLs are properly validated to prevent SSRF. Only allow URLs from legitimate Firebase domains.

3. Sanitize Logs:

- Avoid logging sensitive information directly. Use masked or aggregated logs to minimize information leakage.

4. Handle Failures Gracefully:

- Ensure that failures in `open_firebase()` are properly reflected in `firebase_analysis()` to avoid incorrect analysis results.

By addressing these vulnerabilities, the functions will become more secure and resistant to common web application security risks. Let me know if you need further assistance with code improvements!

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles an attacker might face, and the likelihood of this attack happening. Ensure the exploits are specific to the vulnerability type provided.

Below are exploit examples for the vulnerabilities identified in the `open_firebase()` and `firebase_analysis()` functions. Each exploit is specific to the vulnerability type and includes a step-by-step explanation.

1. Insufficient Error Handling for HTTP Requests

OWASP Category: A09:2021 – Security Logging and Monitoring Failures

Exploit Example:

An attacker forces the application to hang indefinitely or crashes the server by exploiting the lack of a timeout in HTTP requests.

Steps:

1. The attacker identifies the application calling `open_firebase()` with unvalidated Firebase URLs.
2. The attacker submits a Firebase URL pointing to their own server, which does not respond (or sends slow, incomplete responses).
 - Example URL: `https://malicious-firebase.com/.json`
3. The application's `requests.get()` call in `open_firebase()` does not include a timeout and hangs indefinitely, consuming server resources.

Necessary Conditions:

- The application must accept and use attacker-controlled URLs.
- The server running the application must have limited resources or no proper safeguards against long-running processes.

Limitations:

- If the server has a global timeout for HTTP requests or monitors hanging processes, this attack may be mitigated.

Likelihood:

- **Medium:** This is a common misconfiguration, and attackers often exploit such DoS opportunities.

2. Missing Timeout in HTTP Requests

OWASP Category: A04:2021 – Insecure Design

Exploit Example:

An attacker uses slow HTTP responses to exhaust server resources.

Steps:

1. The attacker provides a URL to a server they control.
2. The malicious server responds very slowly, sending only a few bytes at a time to keep the connection open.
3. Since the `requests.get()` call in `open_firebase()` does not specify a timeout, the server processing this request becomes tied up, consuming CPU and memory.

Necessary Conditions:

- The server must process the attacker's URL without implementing a timeout.
- No external monitoring or resource-limiting mechanism is in place.

Limitations:

- A global timeout mechanism (if present) could thwart this attack.

Likelihood:

- **High:** Timeout misconfigurations are prevalent in applications.
-

3. Insufficient Validation of Firebase URLs (SSRF)

OWASP Category: A10:2021 – Server-Side Request Forgery (SSRF)

Exploit Example:

An attacker tricks the application into making requests to internal resources (e.g., a private admin panel).

Steps:

1. The attacker submits a malicious URL to `firebase_analysis()`, such as:
 - `http://localhost:8080/admin`
2. The `open_firebase()` function validates the URL only using `valid_host(url)` but does not ensure it's a legitimate Firebase URL.

3. The application sends an HTTP request to the attacker's specified endpoint (`http://localhost:8080/admin`) under the assumption it is a Firebase URL.
4. If the internal server contains sensitive information or misconfigured endpoints, the attacker can exfiltrate data.

Necessary Conditions:

- The server must accept and process attacker-controlled URLs.
- The application must have access to internal resources not normally exposed to the attacker.

Limitations:

- If the server has a strict URL validation mechanism, this attack becomes impossible.

Likelihood:

- **High:** SSRF is one of the most exploited vulnerabilities.
-

4. Logging of Potentially Sensitive URLs

OWASP Category: A09:2021 – Security Logging and Monitoring Failures

Exploit Example:

An attacker retrieves sensitive data from logs due to careless logging practices.

Steps:

1. The attacker submits a Firebase URL containing a sensitive query string or payload, such as:
 - `https://firebaseio.com/userdata.json?token=abcd1234`
2. The `firebase_analysis()` function logs the URL directly:
 - `logger.info('Detecting Firebase URL(s)')`
3. If logs are not properly secured, an attacker or malicious insider could access them and retrieve sensitive data (`token=abcd1234`).

Necessary Conditions:

- Logs must be accessible to unauthorized users or lack encryption.

- Sensitive data is included in the logged information.

Limitations:

- If logs are encrypted and access-controlled, this attack is less likely.

Likelihood:

- **Medium:** Log mismanagement is common but depends on specific configurations.
-

5. Lack of Error Handling for `open_firebase()` Results

OWASP Category: A04:2021 – Insecure Design

Exploit Example:

An attacker causes the application to malfunction by forcing it to process an invalid Firebase response.

Steps:

1. The attacker provides an invalid Firebase URL that results in a non-200 HTTP response, such as:
 - `https://firebaseio.com/fakeendpoint.json`
2. The `open_firebase()` function fails to handle the non-200 response properly and returns `(url, False)`.
3. The `firebase_analysis()` function adds this result to its output without validation:
 - `{ "url": "https://firebaseio.com/fakeendpoint.json", "open": False }`
4. Downstream functionality relying on this data may crash or produce incorrect results.

Necessary Conditions:

- Downstream code must process the results from `firebase_analysis()`.
- No additional validation exists for the results.

Limitations:

- If `firebase_analysis()` properly checks results before passing them downstream, this attack would fail.

Likelihood:

- **Low:** This is a more indirect attack requiring specific conditions.
-

6. Weak URL Validation in `firebase_analysis()`

OWASP Category: A10:2021 – SSRF

Exploit Example:

An attacker injects a crafted URL that passes weak validation but points to a malicious server.

Steps:

1. The attacker submits a URL like `https://firebaseio.com.attacker.com`.
2. The weak validation in `firebase_analysis()` (`if 'firebaseio.com' in url`) allows the URL to pass.
3. The application connects to the attacker's server (`attacker.com`) instead of a legitimate Firebase endpoint.
4. The attacker's server can then exploit other vulnerabilities, such as leaking data or launching further attacks.

Necessary Conditions:

- URL validation must only check for substring matches.
- The application must allow arbitrary URLs without additional checks.

Limitations:

- If the server uses stricter validation (e.g., `urlparse` with domain checks), this attack would fail.

Likelihood:

- **High:** Substring matching for URL validation is a common mistake.
-

Summary of Exploits

Vulnerability	Exploit Example	Likelihood
Insufficient Error Handling for Requests	Server hangs due to attacker-controlled slow responses or lack of proper error handling.	Medium
Missing Timeout in HTTP Requests	Attacker causes DoS by sending slow or incomplete HTTP responses to exhaust server resources.	High
Insufficient URL Validation (SSRF)	Attacker accesses internal resources or exfiltrates sensitive data by submitting malicious URLs.	High
Logging Sensitive URLs	Sensitive data in query strings or tokens is exposed in application logs, accessible to unauthorized users or insiders.	Medium
Lack of Error Handling for Results	Invalid Firebase responses lead to crashes or incorrect behavior in downstream functionality.	Low
Weak URL Validation in <code>firebase_analysis()</code>	Crafted URLs bypass weak validation and direct the application to malicious servers, enabling data exfiltration or further attacks.	High

Mitigation Overview

1. **Timeouts:** Add timeouts for all HTTP requests to prevent hanging connections.
2. **Strict Validation:** Validate URLs to ensure they match expected patterns or domains (e.g., `*.firebaseio.com`).
3. **Secure Logging:** Avoid logging sensitive data directly; sanitize logs to remove or mask sensitive information.
4. **Error Handling:** Ensure errors in HTTP requests or Firebase responses are handled gracefully, with robust fallback mechanisms.
5. **Harden URL Validation:** Use domain matching (`urlparse`) rather than substring checks to prevent SSRF attacks.

These mitigation steps will address the outlined vulnerabilities and significantly reduce the likelihood of successful exploits. Let me know if you need help implementing these defenses!